FIAP ON

DATA VIZ & DATA SCIENCE

ESTIMAÇÃO E DISCRIMINAÇÃO

13

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 - Exemplo de classificador "perfeito" 5
Figura 2 - Exemplo de uma Árvore de Decisão de um classificador 6
Figura 3 - Gráfico de classes separadas por um classificador utilizando-se Árvores de Decisão 7
Figura 4 - Exemplos de diferentes classificadores com Discriminantes 8
Figura 5 - Dataset mtcars 11
Figura 6 - Histograma da variável mpg 12
Figura 7 - Histograma de uma variável com alta concentração de dados no lado esquerdo 13
Figura 8 - Histograma da variável após transformação Log-Linear 14
Figura 9 - Histogramas das 4 variáveis em uma janela gráfica única 16
Figura 10 - Histogramas das 4 variáveis em uma janela gráfica única 16
Figura 11 - Gráfico de dispersão com identificação das classes 18
Figura 12 - Gráfico de comparação entre 2 métodos discriminantes e outros 2 métodos classificadores 21
Figura 13 - Comparativo da acurácia entre os 3 classificadores 23
Figura 14 - Tabela comparativa de prós e contras de KNN, LDA e QDA 24

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Carga e instalação de bibliotecas 11
Código-fonte 2 - Carga e instalação de bibliotecas 13
Código-fonte 3 - Transformação Log-Linear para redistribuição de dados 14
Código-fonte 4 - Geração de 3 histogramas em uma mesma janela gráfica.....15
Código-fonte 5 - Geração do scatter plot com identificação das classes 17
Código-fonte 6 - Geração do gráfico de saída para os 4 classificadores, a fim de compará-los.....19
Código-fonte 7 - Código-fonte adaptado para inclusão do cálculo da acurácia 22

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

ESTIMAÇÃO E DISCRIMINAÇÃO 5

1 INTRODUÇÃO.....5

2 COMO CALCULAR O DISCRIMINANTE NO PYTHON 10

2.1 Conceito.....10

2.2 Base de dados para o exemplo 10

2.3 Premissa de distribuição de dados: transformações de dados.....12

2.4 Análise descritiva do problema de classificação.....15

2.5 Cálculo do LDA e QDA em Python 18

2.6 Análise comparativa entre os diferentes métodos 20

REFERÊNCIAS 25

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

ESTIMAÇÃO E DISCRIMINAÇÃO

1 INTRODUÇÃO

Um problema frequente ao se analisar dados multivariados em problemas de classificação é encontrar uma boa "separação" entre as classes. No universo de algoritmos classificadores, temos uma série de técnicas e algoritmos diferentes que são utilizados para diferentes resultados de classificação.

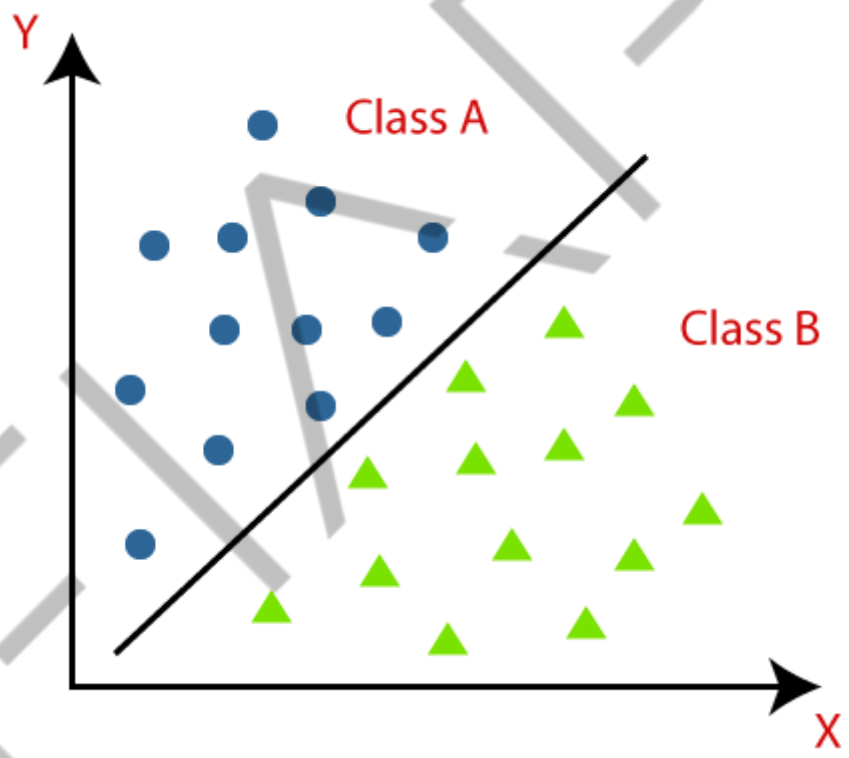

Figura 1 - Exemplo de classificador "perfeito"

Fonte: Elaborado pelo autor (2022)

Por exemplo, já vimos anteriormente um algoritmo chamado de árvores de decisão que pode ser aplicado para problemas de classificação. Ao executarmos um determinado algoritmo, chegamos à árvore apresentada na Figura 2.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

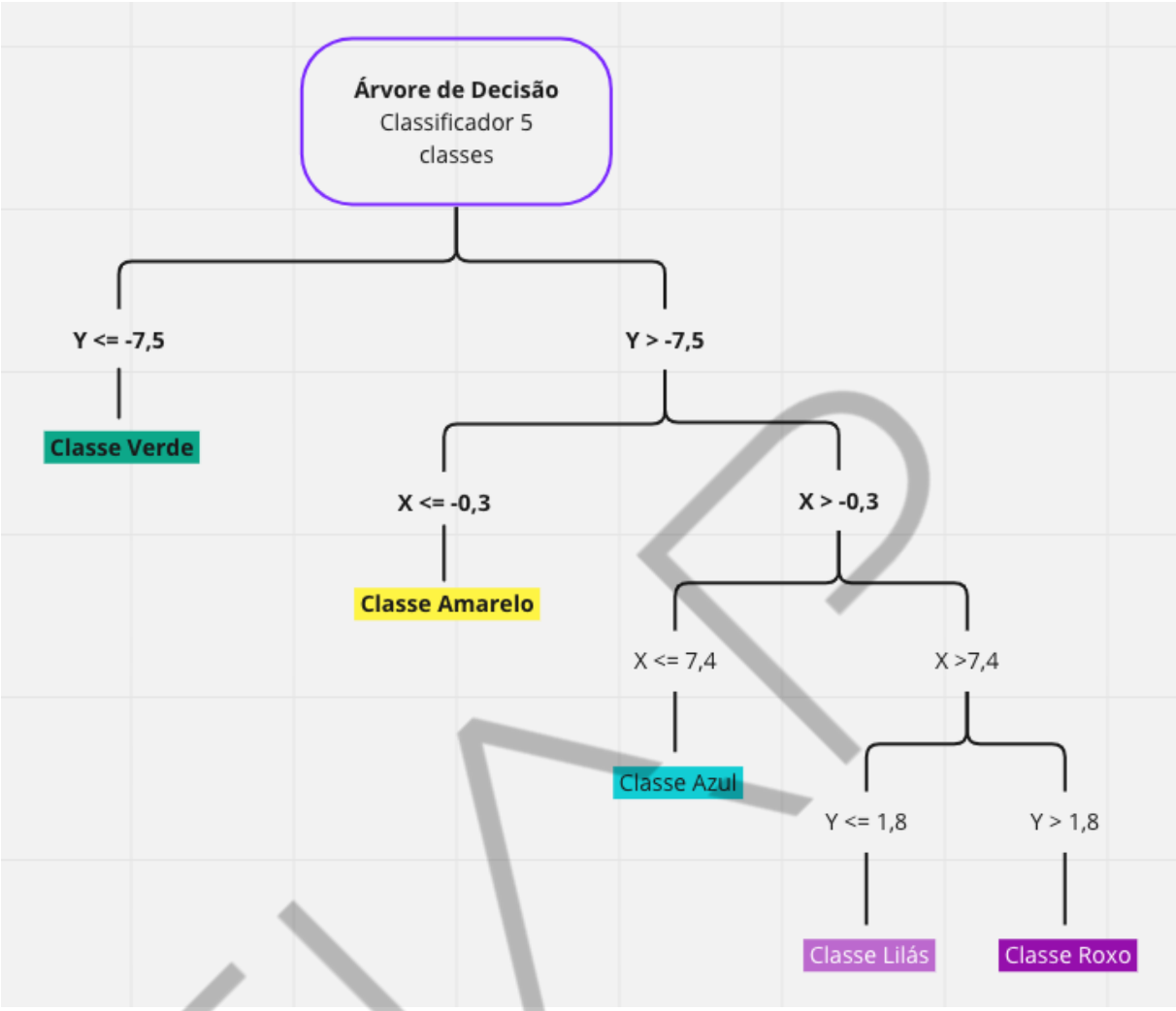

Figura 2 - Exemplo de uma Árvore de Decisão de um classificador

Fonte: Elaborado pelo autor (2022)

Essa árvore pode ser apresentada como resultado das duas variáveis em uma visualização de um gráfico de dispersão[1] apresentado na Figura 3. Como exercício, procure analisar a árvore descrita na Figura 2 em relação ao gráfico apresentado na Figura 3 e verifique se fica o entendimento de que ambas representam a mesma árvore, porém em tipos de gráficos diferentes.

[1] Neste caso foi possível a geração de um gráfico de dispersão, pois estamos utilizando, neste exemplo, duas variáveis somente, a X e Y. Quando utilizado um conjunto maior de variáveis, a ideia continuará sendo exatamente a mesma, porém com muitas variáveis a visualização gráfica fica muito mais complicada. Por isso, é importante ter um referencial em mente para nos auxiliar no processo de análise nas associações e ajustes de algoritmos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

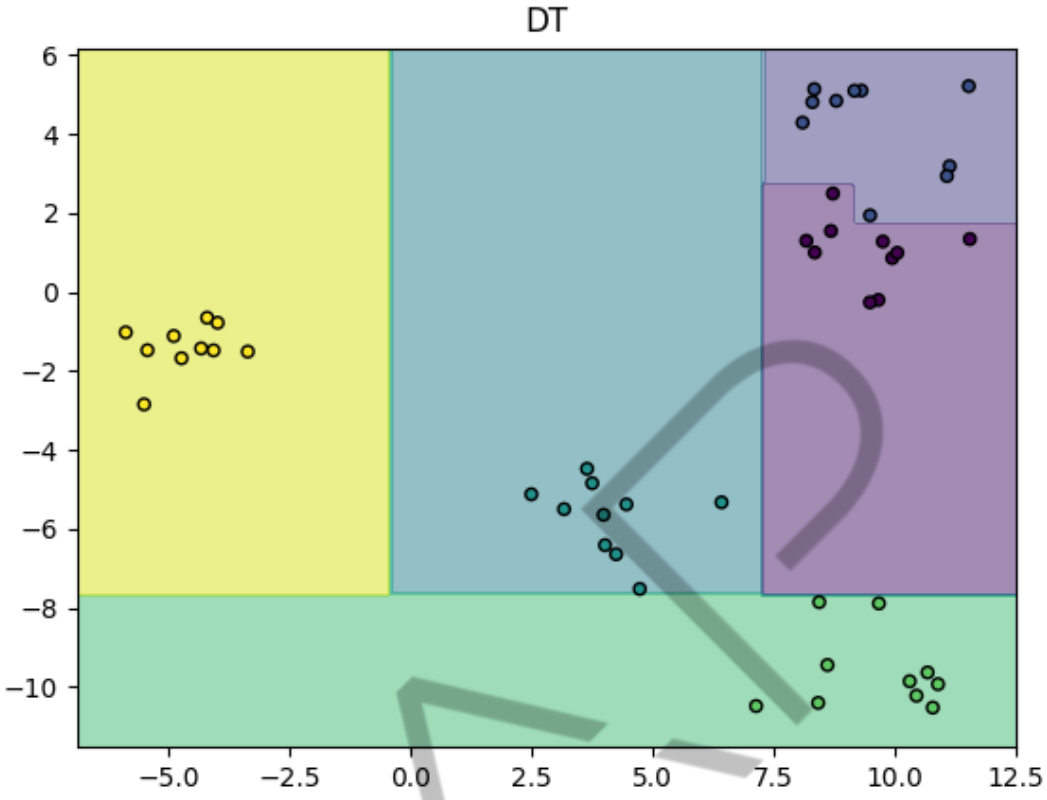

Figura 3 - Gráfico de classes separadas por um classificador utilizando-se Árvores de Decisão
Fonte: https://kssk.gitbook.io/metody-sztucznej-inteligencji/modules/2.-types-of-classifiers

O observador mais atento irá notar que há uma pequena inconsistência na divisão das classes lilás e roxa, pois há um pequeno "degrau" entre elas. Esse degrau, de fato, não está representado na árvore da Figura 2. Mas, exceto por esse fato e por alguma possível aproximação, podemos dizer que é praticamente a mesma representação.

Mas o que queremos mostrar aqui é que uma árvore de decisão como classificador faz "recortes" no formato de blocos ou retângulos e isso diz respeito à natureza do funcionamento de uma árvore de decisão, que é na verdade uma árvore de probabilidades e, para variáveis numéricas, o comparador que se utiliza é > ou <.

Apesar de a árvore de decisão ser um algoritmo bastante útil (além de suas variantes Random Forest, Gradient Boosting Trees, entre outras), nem todos os problemas (de classificação) podem ser resolvidos dessa forma. Assim sendo, o

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

conjunto de técnicas de discriminantes pode ser aplicado justamente aí para criar, no que diz respeito à visualização, geometrias diferentes de separação de classes.

Veremos agora, na Figura 4, quatro tipos de aplicações de discriminantes de tipos diferentes para avaliarmos como se comportam, quando comparados a uma árvore de decisão.

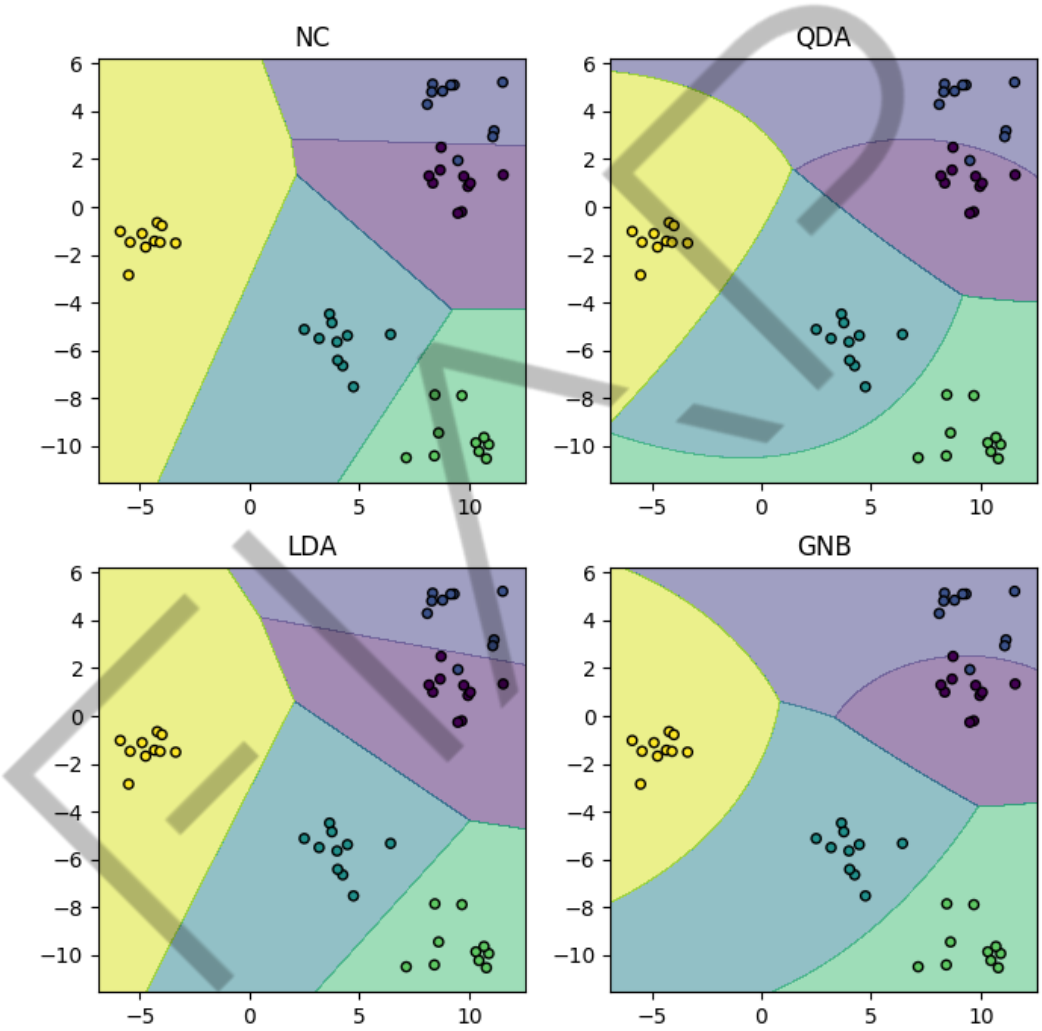

Figura 4 - Exemplos de diferentes classificadores com Discriminantes

Fonte: https://kssk.gitbook.io/metody-sztucznej-inteligencji/modules/2.-types-of-classifiers

Repare que os classificadores NC e LDA mostram um gráfico separado por linhas, pelo que supomos que são discriminantes lineares. Já os QDA e GNB apresentam curvas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Nesse exemplo temos 2 aplicações de discriminantes e 2 outros algoritmos classificadores. Os 2 exemplos da esquerda (1ª coluna) são bem parecidos, mas não iguais. O mesmo ocorre para os exemplos da direita (2ª coluna). Vamos descrever cada um deles:

- NC: abreviação de nearest centroid ou vizinho mais próximo. É um algoritmo bem simples para classificar (portanto, um modelo supervisionado) objetos próximos ou parecidos[2]. Não se utiliza de técnica de discriminante, está postado aqui para efeito comparativo.
- LDA: análise de discriminante linear, objeto de estudo deste capítulo que veremos à frente.
- QDA: é uma variante do LDA, porém com aplicação de uma função quadrática (Q) que nada mais é do que uma potência 2, que como sabemos gera uma curva do tipo parábola (olhe de novo a Figura 4 e imagine vários "pedaços" de parábolas conectados – é exatamente isso que o algoritmo faz, separa classes com pedações de parábolas conectados).
- GNB: o *naive*[3] Bayes gradiente utiliza-se de um algoritmo baseado no teorema de Bayes com uma técnica combinada proposta por Naive. Também não se utiliza de técnica de discriminante, está postado aqui para efeito comparativo.

[2] Esta ideia é bem parecida com a de aglomerados (clusters), porém a diferença entre um K-Means e um KNN é que o primeiro não se utiliza de uma variável alvo como guia, portanto, sendo um método não supervisionado. O segundo necessita de uma variável alvo pré-classificada, sendo, portanto, um método supervisionado (preditivo).

[3] O termo “naive”, em tradução livre, significa “ingênuo”, o que leva a tradução do algoritmo para algoritmo ingênuo de Bayes, que denota sua simplificação.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 COMO CALCULAR O DISCRIMINANTE NO PYTHON

2.1 Conceito

Basicamente, a ideia de um discriminante é obter uma combinação linear (de variáveis numéricas) que melhor represente a discriminação entre diferentes tipos (classes) de uma variável. Em geral é aplicável a duas ou às classes de uma variável alvo que se quer discriminar.

A técnica é classificada como uma estatística multivariada (JOHNSON & WICHNER) que é composta inicialmente por uma etapa discriminação (ou separação), sendo a parte da análise em que exploramos os dados a fim de buscar características (variáveis exógenas) que melhor encaixem os objetos (observações de uma mesma classe) em suas respectivas classes.

2.2 Base de dados para o exemplo

Para seguir nossos estudos, vamos utilizar uma base de dados, já utilizada anteriormente, composta de 24 veículos, com os seguintes dados:

- mpg: milhas por galão (o equivalente, em unidades de medidas utilizadas no Brasil, seria quilômetros por litro). Esta medida dá uma ideia do inverso do consumo de veículos, isto é, veículos com mpg maior são mais econômicos e mpg menor são veículos que consomem mais combustível;
- wt: é a abreviação para peso, na unidade de medida toneladas;
- cyl: a quantidade de cilindros[4] dos motores dos veículos.

Vejamos uma tabela com os dados presentes nesse dataset, para recapitularmos, na Figura 5.

[4] Você irá notar que vários veículos têm 6 ou ainda 8 cilindros. Essa base de dados é de veículos americanos da década de 70 quando era comum motores maiores, muito potentes, e que consumiam muito. Atualmente, a maioria de veículos à combustão possui 4 cilindros, sendo que muitos modernos têm 3 cilindros.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	Unnamed: 0	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
0	Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
1	Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
2	Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
3	Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
4	Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
5	Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
6	Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
7	Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
8	Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
9	Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
10	Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
11	Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3

Figura 5 - Dataset mtcars

Fonte: Elaborado pelo autor (2022)

Em nossos exemplos neste capítulo, procuraremos analisar a quantidade de cilindros do motor de veículos, sendo que procuraremos determiná-la a partir das outras duas propriedades. Isto é, mpg será a nossa variável alvo e wt e mpg serão as variáveis preditoras.

Vamos, inicialmente, conhecer nossa base de dados por meio de análise descritiva, iniciando no código-fonte 1 com a carga das bibliotecas que utilizaremos, carga dos dados e histograma da 1ª variável, mpg.

```
import pandas as pd
import numpy as np
df =
pd.read_csv("https://raw.githubusercontent.com/diogenesjusto/FIAP-On-
Line-Graduation/main/datasets/%20mtcars.csv")
df['mpg'].hist()
```

Código-fonte 1 - Carga e instalação de bibliotecas

Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O histograma gerado pode ser visualizado na Figura 6, onde podemos notar que a variável mpg tem uma distribuição relativamente boa, ou seja, os valores da variável não estão todos acumulados ou agrupados em torno de somente uma pequena região.

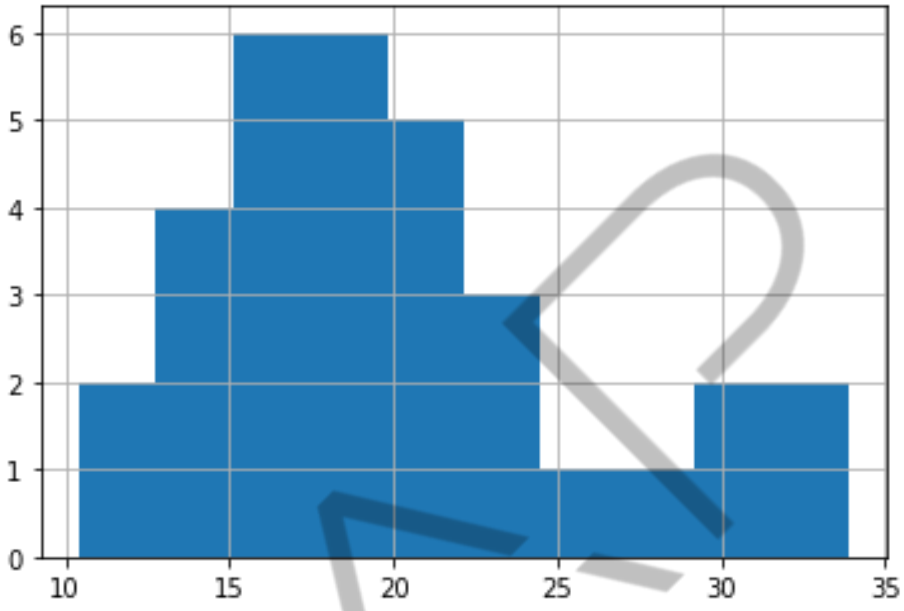

Figura 6 - Histograma da variável mpg

Fonte: Elaborado pelo autor (2022)

2.3 Premissa de distribuição de dados: transformações de dados

Se esse fosse o caso (uma aglomeração de muitos dados em torno de uma região pequena), poderia denotar a necessidade de uma transformação de dados, como etapa intermediária, antes da execução do algoritmo. E isso seria necessário visto que uma classificação de dados poderia tender a ter somente uma região, visto que todos os dados estariam muito próximos.

Quer ver um exemplo? Veja o código-fonte 2.

```
import pandas as pd
import numpy as np
df =
pd.read_csv("https://raw.githubusercontent.com/diogenesjusto/FIAP-On-
Line-Graduation/main/datasets/diamonds.csv")
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
df['price'].hist()
```

Código-fonte 2- Carga e instalação de bibliotecas

Fonte: Elaborado pelo autor (2022)

O histograma gerado por esse código é apresentado na Figura 7, e podemos notar nitidamente que temos uma grande aglomeração de dados no lado esquerdo. Existem casos em que essa concentração fica ainda muito mais evidente.

Se tentarmos utilizar algum separador de dados, um classificador, há uma boa chance de conseguirmos dividir em poucos segmentos, quando não é o caso de termos somente um segmento. Isso acontece pois os dados não estão bem distribuídos. Isso não é, necessariamente, um problema com os dados. Pode se tornar um problema se decidirmos utilizar alguma ferramenta (função matemática) do tipo linear, por exemplo, sem antes redistribuir os dados.

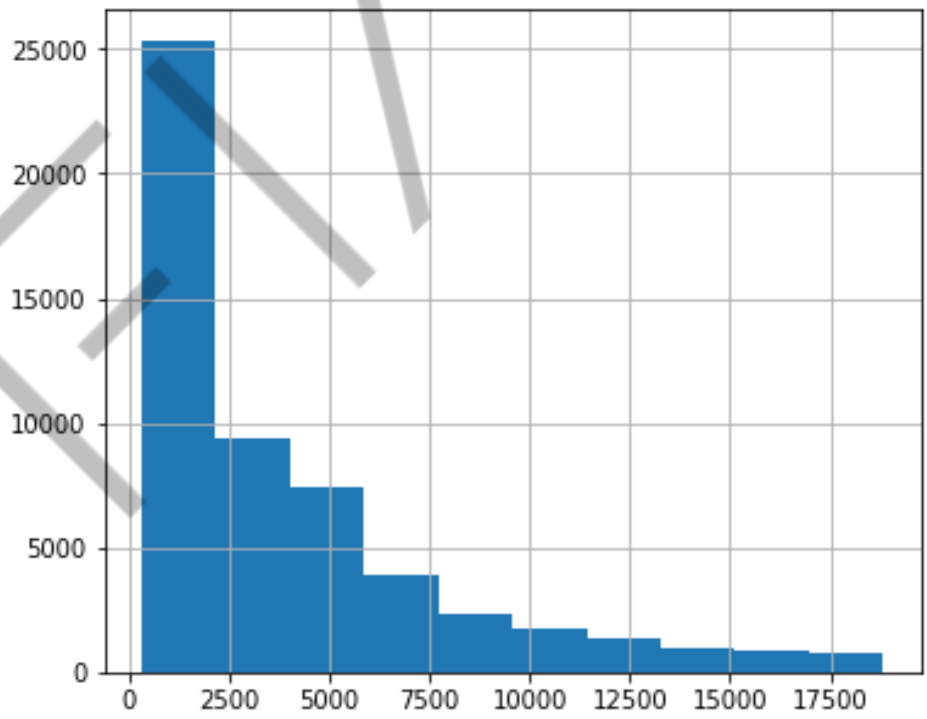

Figura 7 - Histograma de uma variável com alta concentração de dados no lado esquerdo

Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

No caso da Figura 6, existem vários métodos que podemos aplicar, sendo que a transformação log-linear é uma das fortes candidatas. Vamos experimentá-la e ver o que acontece? Veja o código-fonte 3.

```
# Transformação Log-Linear com objetivo de redistribuir os dados
dfDiam['price_log'] = np.log(dfDiam['price'])
dfDiam['price_log'].hist()
```

Código-fonte 3 -Transformação Log-Linear para redistribuição de dados

Fonte: Elaborado pelo autor (2022)

Após a transformação, podemos analisar a nova distribuição de dados com o histograma gerado na nova variável, na Figura 8.

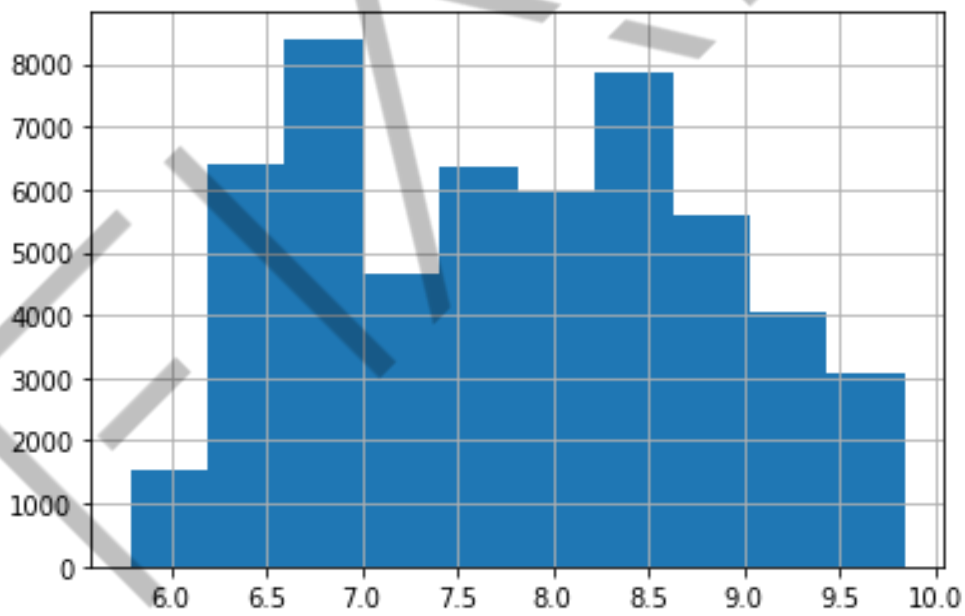

Figura 8 - Histograma da variável após transformação Log-Linear

Fonte: Elaborado pelo autor (2022)

Repare como os dados estão distribuídos de forma mais homogênea, ao longo do eixo x que demonstra seu domínio. Sobre esse histograma algumas perguntas podem aparecer, como: se transformamos os dados, eles não são mais originalmente os mesmos. Será que os cálculos serão conclusivos?

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Devemos lembrar que nos estudos de relações entre variáveis, como análises multivariadas, algoritmos de machine learning, entre outros, estamos sempre analisando a possível relação entre uma variável e outra.

Quando fazemos uma transformação de dados por meio de uma função, não mudamos completamente a variável, somente a apresentamos sob uma ótica diferente, sob uma escala diferente, de tal forma que a parte de "conteúdo" da variável é preservada. É como se estivéssemos esticando, encolhendo, dando *zoom* nos dados.

2.4 Análise descritiva do problema de classificação

Agora que já analisamos a variável mpg e entendemos que em alguns casos pode ser útil (e necessário talvez) uma transformação de dados, vamos conhecer também as distribuições de wt e cyl. Mas… será que não haveria uma forma de gerar os três histogramas lado a lado, de uma vez só, para facilitar a análise, comparando-os? Sim, e pode ser bastante útil. Veja o código-fonte 4.

```
import matplotlib.pyplot as plt
plt.rcParams["figure.figsize"] = (12,5)

fig, axes = plt.subplots(1, 3)

df.hist('mpg', ax=axes[0])
df.hist('wt', ax=axes[1])
df.hist('cyl', ax=axes[2])
```

Código-fonte 4 - Geração de 3 histogramas em uma mesma janela gráfica

Fonte: Elaborado pelo autor (2022)

Utilizamos um recurso da biblioteca gráfica matplotlib, por meio do método rcParams, quando definimos um tamanho diferente para a janela gráfica (os três histogramas juntos em uma janela quadrada, ficariam bem espremidinhos). Veja o resultado na Figura 9.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

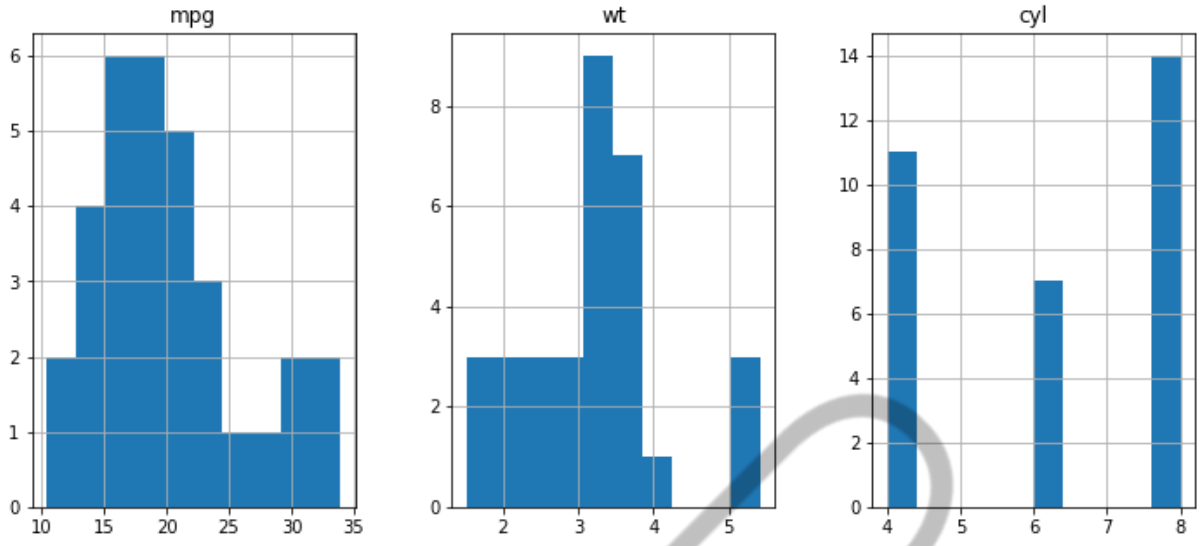

Figura 9 - Histogramas das 4 variáveis em uma janela gráfica única

Fonte: Elaborado pelo autor (2022)

Vamos assumir que temos uma boa distribuição para as três variáveis, de tal forma que conseguiremos fazer "recortes" para identificar as três classes. Nesse caso, as três classes que procuraremos identificar serão os valores da variável cyl, ou seja, veículos de 4, 6 ou 8 cilindros. E, para isso, analisaremos, inicialmente, as variáveis mpg e cyl. Podemos dar o próximo passo analisando a relação entre as variáveis mpg e wt, identificando no gráfico de dispersão os veículos de 4, 6 e 8 cilindros.

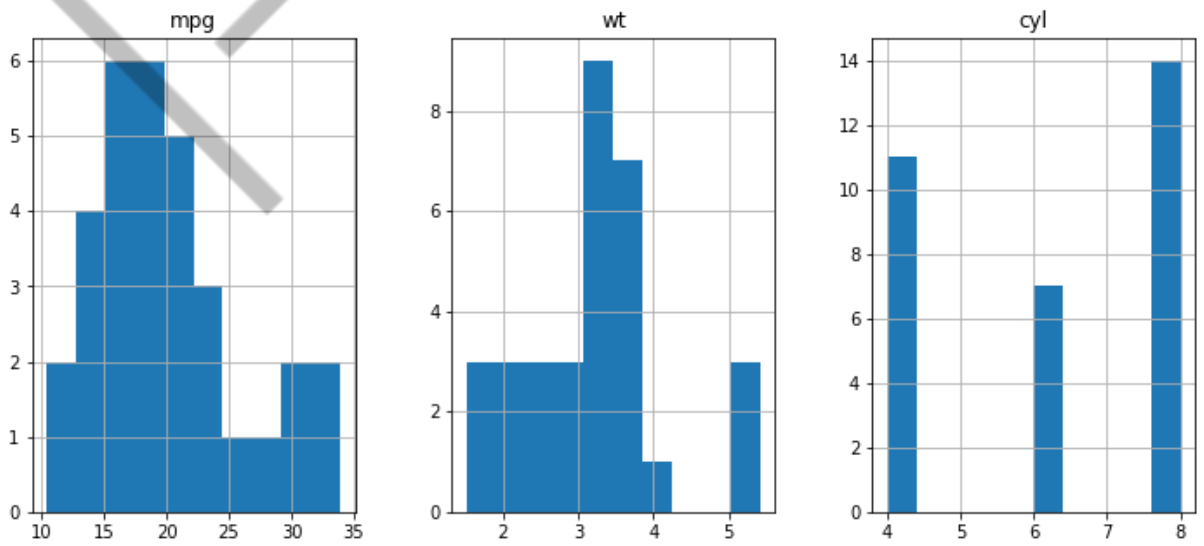

Figura 10 - Histogramas das 4 variáveis em uma janela gráfica única

Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Veja no código-fonte 5 a geração do gráfico de dispersão, onde se procurou identificar as classes de 4, 6 e 8 cilindros utilizando-se do parâmetro c (color).

```
#Gráfico de dispersão entre mpg e wt, identificando os veículos de 4,
6 e 8 cilindros
df['cyl'] = df['cyl'].astype('category')
df.plot.scatter(x="wt", y="mpg",c="cyl", colormap='viridis')
```

Código-fonte 5 - Geração do scatter plot com identificação das classes

Fonte: Elaborado pelo autor (2022)

Podemos notar, na Figura 11, que temos praticamente três classes assim localizadas:

- Canto superior esquerdo: veículos mais leves com maior mpg, portanto, veículos mais econômicos, onde se localizam os veículos da classe 4 cilindros;
- Canto inferior direito: o inverso ocorre, veículos mais pesados e também que consomem mais combustível, onde se localizam os veículos da classe 8 cilindros;
- Meio: os veículos de 6 cilindros, que são um meio termo entre as duas classes anteriores, estão justamente aí.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

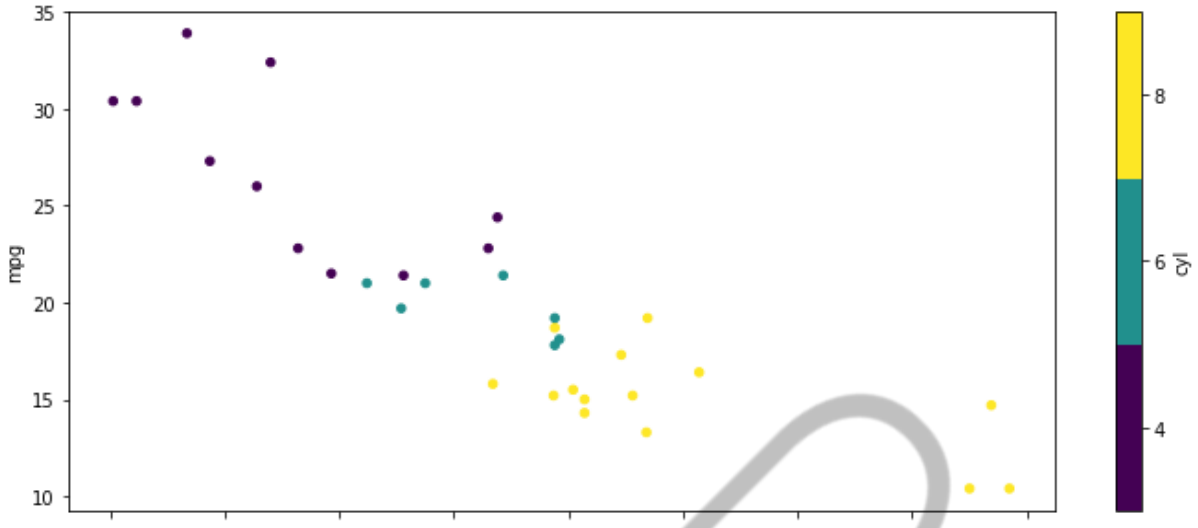

Figura 11 - Gráfico de dispersão com identificação das classes

Fonte: Elaborado pelo autor (2022)

O que vemos no gráfico é bem o que se espera: quando analisamos o que significam as variáveis, que são características de veículos. O que precisamos agora são métodos que nos auxiliem a "separar" as classes, dado que conheçamos somente as variáveis mpg e wt (preditoras).

2.5 Cálculo do LDA e QDA em Python

Vimos, no início do capítulo, uma análise comparativa de quatro métodos para um mesmo conjunto de dados, para o problema de classificação de dados. Vamos aplicar aqui o mesmo método para que consigamos analisar visualmente como os métodos LDA e QDA se comportam para o nosso problema.

```
# Definição das variáveis Y e X (alvo e preditoras)
X = df[['wt', 'mpg']]
y = df['cyl']

# Bloco de configuração da janela gráfica - método mesg grid é útil
para marcar com cores uma região
x_min = df['wt'].min()
x_max = df['wt'].max()
y_min = df['mpg'].min()
y_max = df['mpg'].max()
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                     np.arange(y_min, y_max, 0.01))
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
# Carga das bibliotecas utilizadas pelos 4 classificadores
from sklearn import neighbors, discriminant_analysis, naive_bayes

# Repare que os classificadores LDA e QDA fazem parte da mesma
biblioteca
# porém são métodos diferentes
# A definição abaixo é somente a construção de um objeto ordenado
para,
# posteriormente, invocar os 4 métodos
classifiers = [
    neighbors.NearestCentroid(),
    discriminant_analysis.LinearDiscriminantAnalysis(),
    discriminant_analysis.QuadraticDiscriminantAnalysis(),
    naive_bayes.GaussianNB()
]
classifier_names = ['NC', 'LDA', 'QDA', 'GNB']

# Carga da biblioteca matplotlib e configuração da janela gráfica
import matplotlib.pyplot as plt
fig, ax = plt.subplots(2,2, figsize=(8,8))

# Laço que será executado 4x para:
#   1. Construção do modelo com método .fit
#   2. Aplicação do método predict para uso do modelo
#   3. Métodos para configuração da janela gráfica
for idx, clf in enumerate(classifiers):
    clf.fit(X.values, y)

    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    ax[idx % 2][int(idx/2)].set_title(classifier_names[idx])
    ax[idx % 2][int(idx/2)].contourf(xx, yy, Z, alpha=.5)
    ax[idx % 2][int(idx/2)].scatter(X.iloc[:, 0], X.iloc[:, 1], c=y,
s=20, edgecolor='k')

# E por fim, a exibição do gráfico
plt.savefig('classifiers_1.png')
```

Código-fonte 6 - Geração do gráfico de saída para os 4 classificadores, a fim de compará-los

Fonte: Elaborado pelo autor (2022)

E, para efeito comparativo, mantivemos os outros dois métodos também, que não são métodos de discriminantes, porém também são aplicáveis a problemas de classificação. O código-fonte 6 está bem comentado a cada seção, detalhando o objetivo e o porquê de sua utilização.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.6 Análise comparativa entre os diferentes métodos

Na Figura 12, podemos visualizar 4 gráficos de dispersão que representam todos os mesmos pontos do nosso data-set, com cores para cada categoria diferente, porém com a diferença de que cada um dos gráficos representa um método diferente como tentativa de criar um classificador.

E como podemos avaliar os classificadores?

As três regiões são o "produto" do nosso classificador, ou seja, o nosso modelo classificador. Basicamente a região superior, em todos os gráficos, diz respeito a veículos de 4 cilindros. Quando utilizarmos cada um dos nossos modelos para realizar uma previsão, a região hachurada superior é que "marcará" um veículo como sendo de 4 cilindros. Imaginemos assim:

1. Construímos um modelo, que pode ser o LDA, para classificar veículos como sendo de 4, 6 ou 8 cilindros, sem abrir o "capô" do veículo, somente sabendo seu peso e consumo;
2. Ao escolhermos um veículo fora da nossa amostra de treino, e conhecemos seu peso e consumo, posicionaremos graficamente o ponto utilizando o gráfico de LDA.
3. Se o veículo tiver 4 cilindros mesmo e for marcado na região hachurada em lilás, significa que nosso modelo acertou a previsão. Caso contrário, será um erro.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

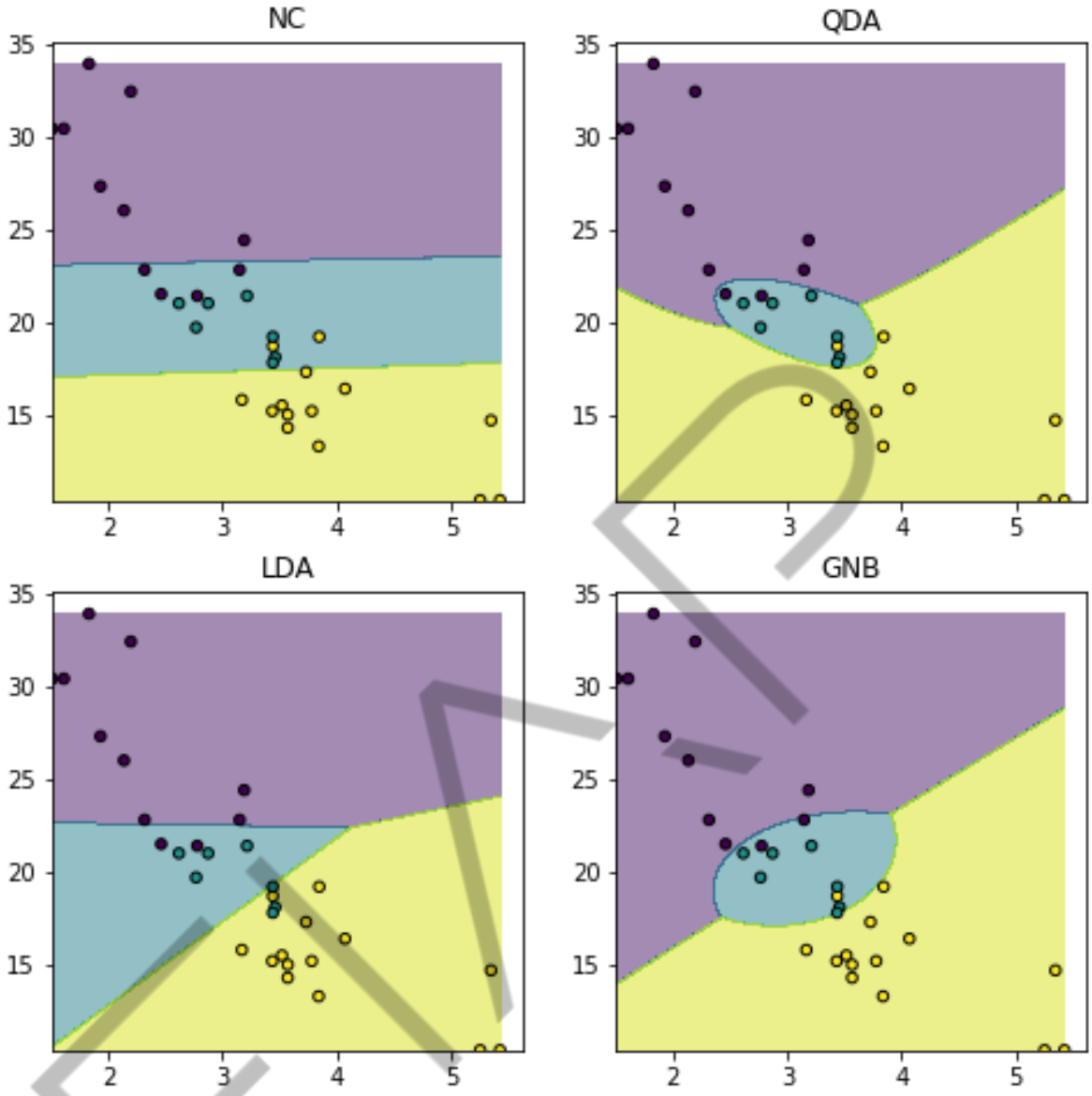

Figura 12 - Gráfico de comparação entre 2 métodos discriminantes e outros 2 métodos classificadores
Fonte: Elaborado pelo autor (2022)

Pode parecer meio sem sentido fazer uma previsão do número de cilindros de um veículo, não é mesmo? O objetivo aqui é utilizar a base de dados conhecida, de uma forma didática, para que fique bem claro o que está sendo proposto. O mesmo classificador poderia ser portado para uma aplicação de churn, fraude, risco de crédito etc.

Mas como avaliamos, por meio desse gráfico, qual é o "melhor" classificador? Em termos numéricos, a acurácia seria representada pela quantidade de pontos de veículos de uma determinada classe que ficaram "dentro" da região correta. Ou seja,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

neste caso utilizando-se de cores, verifique quantos pontos caíram "fora" da sua "cor de origem". Quanto mais pontos fora, pior é o modelo.

É bem interessante analisar graficamente o resultado de um modelo para se conhecer o seu comportamento. Mas o uso de métricas, neste caso, se faz necessário para termos um número, uma métrica que nos dê com uma melhor exatidão qual dos, neste caso, quatro modelos, tem uma melhor performance.

Vamos adicionar o cálculo da acurácia no código acima e comparar os resultados dos quatro diferentes modelos.

```
# Carga das bibliotecas utilizadas pelos 4 classificadores
from sklearn import neighbors, discriminant_analysis, naive_bayes
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Repare que os classificadores LDA e QDA fazem parte da mesma
biblioteca
# posteriormente, invocar os 4 métodos
classifiers = [
    neighbors.NearestCentroid(),
    discriminant_analysis.LinearDiscriminantAnalysis(),
    discriminant_analysis.QuadraticDiscriminantAnalysis(),
    naive_bayes.GaussianNB()
]
classifier_names = ['NC', 'LDA', 'QDA', 'GNB']

# Separação das amostras
X_train, X_test, Y_train, Y_test = train_test_split( df[['wt','mpg']],
df['cyl'], test_size=0.25, random_state=33 )

# Laço que será executado 4x para:
#   1. Construção do modelo com método .fit
#   2. Aplicação do método predict para uso do modelo
#   3. Avalicão de acurácia
for idx, clf in enumerate(classifiers):
    clf.fit(X_train.values, Y_train)


    Y_prev = clf.predict(X_test.values)
    print(classifier_names[idx], ' => acurácia= ',
accuracy_score(Y_test, Y_prev))
```

Código-fonte 7 - Código-fonte adaptado para inclusão do cálculo da acurácia
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

O código-fonte 7 foi uma adaptação do código-fonte 6, porém com objetivo de trazer uma tabela comparativa da métrica Acurácia, entre os quatro modelos classificadores. Vejamos os ajustes explicados abaixo:

- Para avaliar treino e teste, ou seja, um pipeline de Machine Learning para esses classificadores, é necessário separar as amostras. Utilizamos aqui a biblioteca train_test_split;
- Como vamos calcular a acurácia, escolhemos a função implementada no sklearn chamada accuracy_score;
- Realizamos a separação das amostras de treino e teste, de forma aleatória, visto que o dataset é uma "fotografia" dos dados, com tamanhos de base treino 75% e 25% para teste;
- Por fim, rodamos o laço (for) em cada iteração calculando a acurácia.

A figura 13 mostra o resultado comparativo dos quatro algoritmos classificadores após a execução.

```
NC  => acurácia=  0.75
LDA  => acurácia=  0.75
QDA  => acurácia=  0.875
GNB  => acurácia=  0.75
```

Figura 13 - Comparativo da acurácia entre os 3 classificadores
Fonte: Elaborado pelo autor (2022)

A figura 11 é a tabela de métricas do que é demonstrado visualmente na Figura 9. Uma forma de realizar a interpretação, como citado anteriormente, é contar quantas "bolinhas" de uma determinada cor estão de fato na região hachurada de cor correspondente.

No artigo denominado "Evaluating KNN, LDA and QDA Classification for embedded online Feature Fusion", [STARZACHER & RINNER] utilizam um caso

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

prático para comparar os algoritmos que citamos acima, com alguns resultados interessantes de prós e contras, demonstrados na tabela da Figura 14.

Method	Pros	Cons
KNN	notable classification results no (re)training phase distance metrics error probability bounded	time consuming classification time memory utilization finding optimal k
LDA	linear decision boundary fast classification easy to implement	Gaussian assumptions training time complex matrix ops
QDA	quadratic decision boundary fast classification classification more accurate outperforms KNN and LDA	Gaussian assumptions training time complex matrix ops

Figura 14 - Tabela comparativa de pros e contras de KNN, LDA e QDA

Fonte: Elaborado pelo autor (2022)

Em resumo, podemos observar que o LDA e o QDA têm na performance sobre bases maiores um trunfo em relação ao KNN. No entanto, a premissa gaussiana, isto é, dados com distribuição normal, pode ser um problema ao aplicar LDA e QDA. Em geral o QDA, por causa de suas geometrias curvas (quadráticas) em uma performance final em modelos, é melhor.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

HASTIE, T., TIBSHIRANI, R., FRIEDMAN J. ***The Elements of Statistical Learning.*** Section 4.3, p.106-119, 2008.

JOHNSON, R. A.; WICHERN, D. W. ***Applied multivariate statistical analysis. 4th***. 4. ed. New Jersey: Prentice-Hall, 1999.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com